

Shared Code Across Repositories

A scheme built on `drwho`, with `dihiggs` as the worked example

Harrison B. Prosper

Department of Physics, Florida State University

Summary

Common code that had been copied into several repositories now lives in one package, `drwho`. Dependent repositories declare it as a PEP 508 direct reference to its GitHub URL, so a user who clones a repository and runs `pip install -e .` gets everything, with no extra step and nothing to explain. This document records how that is set up, why each piece is the way it is, and—most importantly—the protocol for shipping a fix so that it actually reaches the people using it.

Contents

1	The problem	1
2	The scheme	1
3	The shared repository: drwho	1
3.1	Layout	1
3.2	pyproject.toml	2
3.3	One place for the version number	3
3.4	Lazy submodule loading	3
4	A dependent repository: dihiggs	3
4.1	Layout after migration	3
4.2	Verifying that nothing is lost	4
4.3	What changed in the notebooks	4
5	Installing	5
5.1	Laptop or cluster	5
5.2	Colab	5
5.3	Keep both repositories public	5
6	The release protocol	5
6.1	The rule	6
6.2	The loop	6
6.3	Choosing the number	6
6.4	Tags	6
6.5	The failure mode to recognise	7
6.6	Reproducibility across semesters	7
7	Migrating another repository	7
8	Troubleshooting	8
A	Reference: the two pyproject.toml files	8
A.1	drwho	8
A.2	dihiggs	8
B	Reference: .gitignore additions	9

1 The problem

Several repositories under `github.com/hbprosper` carried byte-identical copies of the same utility modules. That duplication was deliberate: it made each repository self-contained, so a student could clone one thing and run it without first assembling an environment. The cost is the usual one. A fix applied in one copy does not reach the others, the copies drift apart, and after a while nobody can say which version is authoritative.

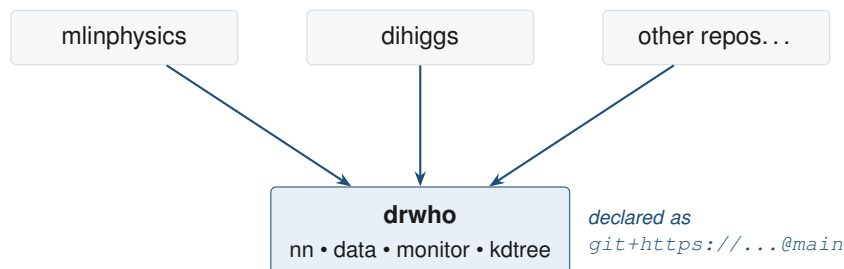
The goal is to remove the duplication *without* removing the property that made it attractive. A student should still clone one repository, run one command, and have a working environment. They should not need to know that a second repository exists.

2 The scheme

One repository, `drwho`, holds the shared code. Every dependent repository names it in the `dependencies` list of its `pyproject.toml`, not by version but by URL:

```
[project]
name = "dihiggs"
dependencies = [
    "drwho @ git+https://github.com/hbprosper/drwho.git@main",
    "numpy",
    "pandas",
    "matplotlib",
    "torch",
]
```

That is a PEP 508 *direct reference*. When pip installs the dependent package it clones `drwho`, builds it, and installs it like any other dependency. The user sees one extra line scroll past.



Why not PyPI?

Publishing to PyPI would also work, and would be slightly smoother still—no git needed at install time, and proper version resolution. It was rejected here because this code is not intended to be “official”. The consequence is worth knowing: PEP 508 direct references are forbidden in packages uploaded to PyPI, so adopting this scheme forecloses publishing these particular repositories later without changing the dependency line first.

3 The shared repository: `drwho`

3.1 Layout

```
drwho/
|-- pyproject.toml
|-- README.md
|-- LICENSE
|-- .gitignore
```

```

|-- docs/
`-- drwho/
    |-- __init__.py      <- version lives here; lazy submodule loading
    |-- data.py
    |-- kdtree.py
    |-- monitor.py
    |-- nn.py
    `-- bin/
        |-- __init__.py
        `-- monitor_losses.py  <- the "monlosses" console script

```

The repository and the package share a name, which is the usual convention and makes the path from `pip install` to `import` obvious.

3.2 pyproject.toml

```

[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "drwho"
dynamic = ["version"]
description = "Shared utilities for Harrison Prosper's physics and ML course
↳ repositories"
readme = "README.md"
requires-python = ">=3.10"
license = "MIT"
license-files = ["LICENSE"]
authors = [{name = "Harrison B. Prosper", email = "hprosper@fsu.edu"}]
dependencies = [
    "numpy", "scipy", "pandas", "matplotlib",
    "torch", "requests", "pyyaml",
]

[project.optional-dependencies]
tensorboard = ["tensorboard"]

[project.urls]
Homepage = "https://github.com/hbprosper/drwho"

[tool.setuptools]
packages = ["drwho", "drwho.bin"]

[tool.setuptools.dynamic]
version = {attr = "drwho.__version__"}

[project.scripts]
monlosses = "drwho.bin.monitor_losses:main"

```

Four points deserve comment.

Table order matters in TOML. Everything written after a `[table]` header belongs to that table. A `dependencies` list placed after `[project.urls]` silently becomes a key *inside* that table, and is never seen as a dependency list at all. Keep every bare key of `[project]` above the first sub-table header.

Dependencies are unpinned, and that is deliberate. Colab ships a `torch` build matched to its own CUDA driver, and so does the university cluster. An upper bound such as `torch>=2.4,<2.6`

would, on the wrong day, make pip download several gigabytes mid-class, possibly landing a CPU-only or driver-mismatched build and demanding a runtime restart. With no bound, pip sees the requirement already satisfied and skips it entirely.

packages is explicit. Left to auto-discovery with `where = ["."]`, `setuptools` will sweep up any top-level directory it mistakes for a package. In `dihiggs` it had recorded `Kurtis`, `code`, `data` and `docs` as installed top-level names. Naming the packages explicitly avoids this, and a subpackage such as `drwho.bin` must be listed—otherwise it is not shipped, and the console script that points into it is dead on arrival.

A subpackage needs `__init__.py`. `drwho/bin/__init__.py` exists so that `drwho.bin` is a regular package. Without it an editable install may still work, via the namespace-package fallback, while a normal wheel install quietly omits the directory—a difference that will not show up until somebody installs it the ordinary way.

3.3 One place for the version number

`pyproject.toml` declares `dynamic = ["version"]` and reads the value from the package itself. The number therefore lives in exactly one place:

```
# drwho/__init__.py
__version__ = "0.2.0"
```

`setuptools` extracts this by parsing the syntax tree, so nothing is imported and `torch` is not needed at build time. Since the version must be bumped on *every* release (Section 6), having two places to edit would be a standing invitation to get them out of step.

3.4 Lazy submodule loading

`drwho/__init__.py` implements PEP 562 module-level `__getattr__`:

```
import importlib as _importlib

__version__ = "0.2.0"
__all__ = ["data", "kdtree", "monitor", "nn"]

def __getattr__(name):
    """Import drwho.<name> on first access (PEP 562)."""
    if name in __all__:
        return _importlib.import_module(f".{name}", __name__)
    raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

So `import drwho` costs essentially nothing and does not drag in `torch`; `drwho.nn` resolves on first touch. Both from `drwho import nn` and `import drwho.nn` continue to work unchanged.

The same principle applies inside `monitor.py`, where `torch.utils.tensorboard` is imported by a helper rather than at module level. An optional dependency should be probed when it is asked for, not announced to everyone who imports the module.

4 A dependent repository: `dihiggs`

4.1 Layout after migration

```
dihiggs/
|-- pyproject.toml
```

```
|-- README.md  LICENSE  .gitignore
|-- code/      <- notebooks, run from here
|  |-- heft_prepare_data.ipynb
|  |-- heft_test.ipynb
|  |-- heft_train.ipynb
|  '-- figures/.gitkeep
|-- data/      <- tracked input data
|-- docs/
|-- clutter/   <- git-ignored; holds retired files pending deletion
'-- dihiggs/
    |-- __init__.py
    '-- heftnet.py    <- the physics: HEFTNet, FEATURES, TARGET
```

What remains in the `dihiggs` package is what is genuinely specific to `dihiggs`. Everything generic moved to `drwho`: 1 120 lines of duplicated code deleted.

4.2 Verifying that nothing is lost

Before deleting a duplicated module, confirm that the shared copy is a superset. Comparing file sizes is not enough; compare the *interface*. Parsing both files and comparing every top-level function and class, and every method within each class, gave for `dihiggs`:

module	identical	body differs	only local	only in <code>drwho</code>
<code>nn.py</code>	11	2	<code>elapsed_time</code>	4
<code>data.py</code>	3	0	—	2
<code>monitor.py</code>	2	3	—	2

No method existed only in the local copies. Every signature difference was an added keyword argument with a default—`Config` gained `dirpath` and `makedirs`, `Monitor` gained `use_tensorboard` and `same_line`, `terminate` gained `delay`—so existing call sites were unaffected. The single local-only function, `elapsed_time`, had moved to `drwho.monitor` and was not called anywhere.

Do this before deleting anything

Compare interfaces, not line counts. A method present only in the copy you are about to delete is the one thing that will break, and it will break later, in someone else's hands.

4.3 What changed in the notebooks

Because the module aliases were kept identical, not one line of notebook body code had to change. Only the import lines moved:

before	after
<code>import dihiggs.nn as mlp</code>	<code>import drwho.nn as mlp</code>
<code>import dihiggs.utils.data as dat</code>	<code>import drwho.data as dat</code>
<code>import dihiggs.utils.monitor as mon</code>	<code>import drwho.monitor as mon</code>
<code>import heftnet as NN</code>	<code>from dihiggs import heftnet as NN</code>

Five lines across three notebooks. The last of these also removed a `sys.path.insert(0, '.')` hack: once `dihiggs` is a properly installed package, `heftnet` is reachable by import from any working directory.

Keep the aliases

Choosing `mlp`, `dat`, `mon` to match the old names turns a migration into a search-and-replace on import lines. It is worth some awkwardness in the alias names to get this.

5 Installing

5.1 Laptop or cluster

```
git clone https://github.com/hbprosper/dihiggs
cd dihiggs
pip install -e .
```

`pip` reads the direct reference, clones `drwho`, builds it, installs it, and then installs `dihiggs` in editable mode. `drwho` lands non-editable in `site-packages`, which is the right default: it is shared code, not something a student should be editing in place.

5.2 Colab

Git installs work normally on Colab. Do *not* clone and then `pip install -e .`; install the whole thing from the URL in one line, and let the direct reference pull `drwho` transitively. As the first cell of every notebook:

```
import sys, subprocess
IN_COLAB = "google.colab" in sys.modules
if IN_COLAB:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                    "dihiggs @ git+https://github.com/hbprosper/dihiggs.git@main"],
                    check=True)
```

This is a no-op locally and on the cluster, where the package is already installed. Two Colab-specific points:

- Every Colab runtime starts empty, so `pip` clones fresh each time. The stale version problem described in Section 6 is a *local-install* problem only; on Colab, `@main` always gives current code.
- Keep the shared repository lean—no data files, no notebooks with stored outputs. It is cloned on every student's every runtime.

5.3 Keep both repositories public

A private repository means students pasting personal access tokens into notebook cells: a hassle, and a bad habit to teach.

6 The release protocol

This is the part that matters. Everything above is set-up done once; this is done every time.

6.1 The rule

The one rule

Bump `__version__` in `drwho/__init__.py` on every release. pip decides whether to reinstall by comparing *version strings*. It does not look at which commit a URL resolves to. A fix pushed without a version bump will not reach anybody, and pip will report success while doing nothing.

This is not folklore. It was measured, with two throwaway packages and a real git dependency, on pip 26.2:

what changed	result of <code>pip install -e . --upgrade</code>
New commit on the shared repo; version left at 0.1.0	stale — old code kept
Pin moved from tag v0.1.1 to tag v0.1.1-hotfix; version string unchanged	stale — old code kept
Version bumped 0.1.0 → 0.1.1	updated (and <code>--upgrade</code> was not even required)

Note the second row. Retagging alone does nothing. Note also the third: once the version changes, a plain `pip install -e .` suffices.

6.2 The loop

```
# --- in drwho ---
# 1. make the change
# 2. bump the version
#     drwho/__init__.py: __version__ = "0.2.0"
git commit -am "Config: add file/config key for the yaml configuration file name"
git tag v0.2.0 # optional; see below
git push && git push --tags

# --- in any dependent repo ---
pip install -e .
# then restart the Jupyter kernel
```

Nothing in the dependent repository is edited. Because it points at `@main` rather than a fixed tag, the dependency line is written once and never touched again. This is the property that makes the scheme worth having: the cost of a fix does not grow with the number of dependent repositories.

6.3 Choosing the number

Semantic versioning, loosely: a backward-compatible fix bumps the patch digit, a new feature—a new keyword argument, a new key in a configuration dictionary—bumps the minor digit. `drwho` went from 0.1.0 to 0.2.0 when `Config` gained its `file/config` key. In the 0.x range nobody will hold you to it; what matters is only that the string *changes*.

6.4 Tags

Tags play no part in the upgrade. Dependent repositories resolve `@main`, so pip never looks at them. Tag anyway, because it costs nothing and gives a fixed point to pin an archived branch to later—but do not expect a tag to deliver a change.

Tags and rebasing

A tag points at a commit hash. If you rebase—for example after editing the README in the GitHub web interface and finding your branches diverged—the rebased commit gets a *new* hash and the tag stays behind on an orphaned one. Delete the tag first, rebase, then re-apply it:

```
git tag -d v0.2.0
git pull --rebase
git tag v0.2.0
git push && git push --tags
```

6.5 The failure mode to recognise

If you edit the shared code but forget the version bump, pip will print `Successfully installed` and leave the old code in place. Nothing errors. The symptom appears later as a fix that “didn’t work”.

When that happens, the hammer is to force the dependency alone:

```
pip install --force-reinstall --no-deps \
    "drwho @ git+https://github.com/hbprosper/drwho.git@main"
```

Use it as a rescue, not as routine. The routine is the version bump.

6.6 Reproducibility across semesters

Pointing at @main means an archived copy of a course repository will pull *today’s* shared code, not the code it was written against. If that matters, cut a frozen branch of `drwho` at the end of each semester (fall-2026) and change the dependency line in the archived copy only. One edit per semester instead of one per fix.

7 Migrating another repository

A checklist, in the order that minimises the chance of a nasty surprise.

1. **Compare interfaces**, not file sizes, between the local copies and `drwho`. Confirm that nothing exists only locally.
2. **Inventory the symbols actually used** by the notebooks and scripts, and check every one against `drwho`.
3. **Move retired files aside** into a git-ignored `clutter/` directory rather than deleting them. Delete by hand later, once the migration is proven.
4. **Rewrite the import lines**, keeping the module aliases identical so that no body code changes.
5. **Fix `pyproject.toml`**: add the `drwho` direct reference, list `packages` explicitly, check that no bare key sits below a sub-table header.
6. **Uninstall the old package** before reinstalling—`pip uninstall -y <name>`—so that a stale editable path hook does not shadow the new layout.
7. **`pip install -e .`**, then a one-line smoke test that exercises the whole chain:

```
python -c "import drwho, dihighs.heftnet as NN; print(drwho.__version__, NN.
    ↪ FEATURES)"
```

8. **Restart the Jupyter kernel** before opening a notebook.

Move, don't delete

Retired files go to `clutter/`, which is listed in `.gitignore`. Git records the deletion from the tracked path; the bytes stay on disk until you are sure. In `dihiggs` this parked 33 MB of stale runs, figures, archives and superseded modules without a single irreversible step.

8 Troubleshooting

The fix didn't take. The version was probably not bumped. See Section 6. Confirm with `python -c "import drwho; print(drwho.__version__)"`.

The notebook still uses the old code. A running Jupyter kernel caches modules in `sys.modules`. Restart it. This is the single most common cause of an upgrade that appears not to have happened.

Imports resolve to the wrong place. An earlier editable install may still have a path hook pointing at a repository root whose layout has changed. `pip uninstall -y <name>` and reinstall.

could not read Username for 'https://github.com'. The shell doing the push has no credentials. Push from a terminal where your credential helper is available. Fetching a public repository needs none.

Divergent branches after a web edit. Editing a file in the GitHub interface creates a commit you do not have. `git pull --rebase` reconciles it; if the two sides touch different files no conflict is possible. Mind the tag (Section 6). `git config --global pull.rebase true` stops git asking each time.

A package name on PyPI is not yours. The direct reference always wins, so installs are unaffected. The risk is a typo: `pip install drwho` without the URL would fetch whatever is on PyPI under that name. The original candidate name for this package, `hal9000`, was an empty 2019 placeholder on PyPI—an install that succeeds and then fails at import, which is worse than an install that fails outright. `drwho` was free. Check before choosing.

A Reference: the two `pyproject.toml` files

A.1 `drwho`

See Section 3.2; it is reproduced there in full.

A.2 `dihiggs`

```
[build-system]
requires = ["setuptools>=77"]
build-backend = "setuptools.build_meta"

[project]
name = "dihiggs"
dynamic = ["version"]
description = "Modeling the di-Higgs differential cross section."
readme = "README.md"
requires-python = ">=3.10"
```

```
license = "MIT"
license-files = ["LICENSE"]
authors = [{ name = "Harrison B. Prosper", email = "hprosper@fsu.edu" }]
dependencies = [
    "drwho @ git+https://github.com/hbprosper/drwho.git@main",
    "numpy",
    "pandas",
    "matplotlib",
    "torch",
]

[project.urls]
Homepage = "https://github.com/hbprosper/dihiggs"

[tool.setuptools]
packages = ["dihiggs"]

[tool.setuptools.dynamic]
version = {attr = "dihiggs.__version__"}
```

B Reference: .gitignore additions

```
# Jupyter
.ipynb_checkpoints/

# Python
__pycache__/
*.py[cod]
*.egg-info/
build/
dist/
.venv/

# macOS
.DS_Store

# Local scratch: files pulled out of the repo, pending manual deletion
clutter/
```

The scheme in one sentence: one shared package, declared by URL, upgraded by changing a version string.